Single Solution-based Metaheuristics
— Local Search —

Presented by Jacomine Grobler

Stellenbosch Unit for Operations Research in Engineering
Department of Industrial Engineering

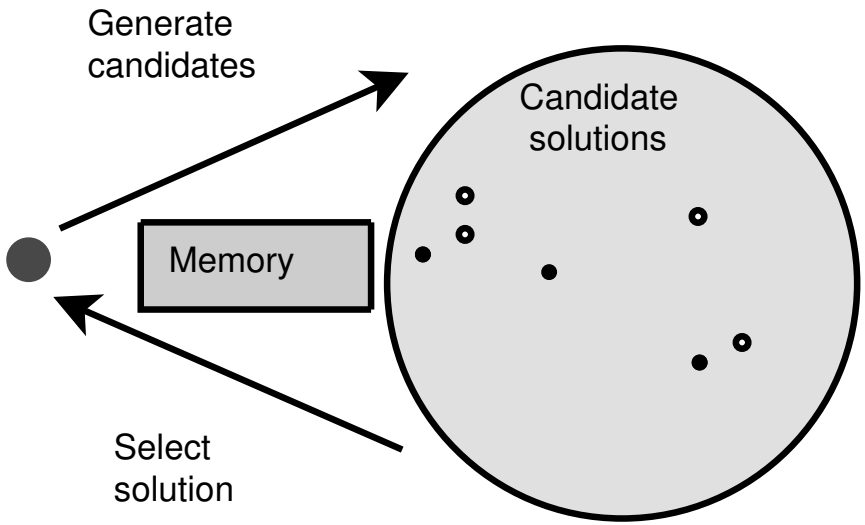
Lecture Contents

(From the textbook by E-G Talbi titled *Metaheuristics: From Design to Implementation*)

Local Search (§2.3 & §2.6)

Local Search Template (pp. 121–123)

Local search algorithmic template (p. 122)



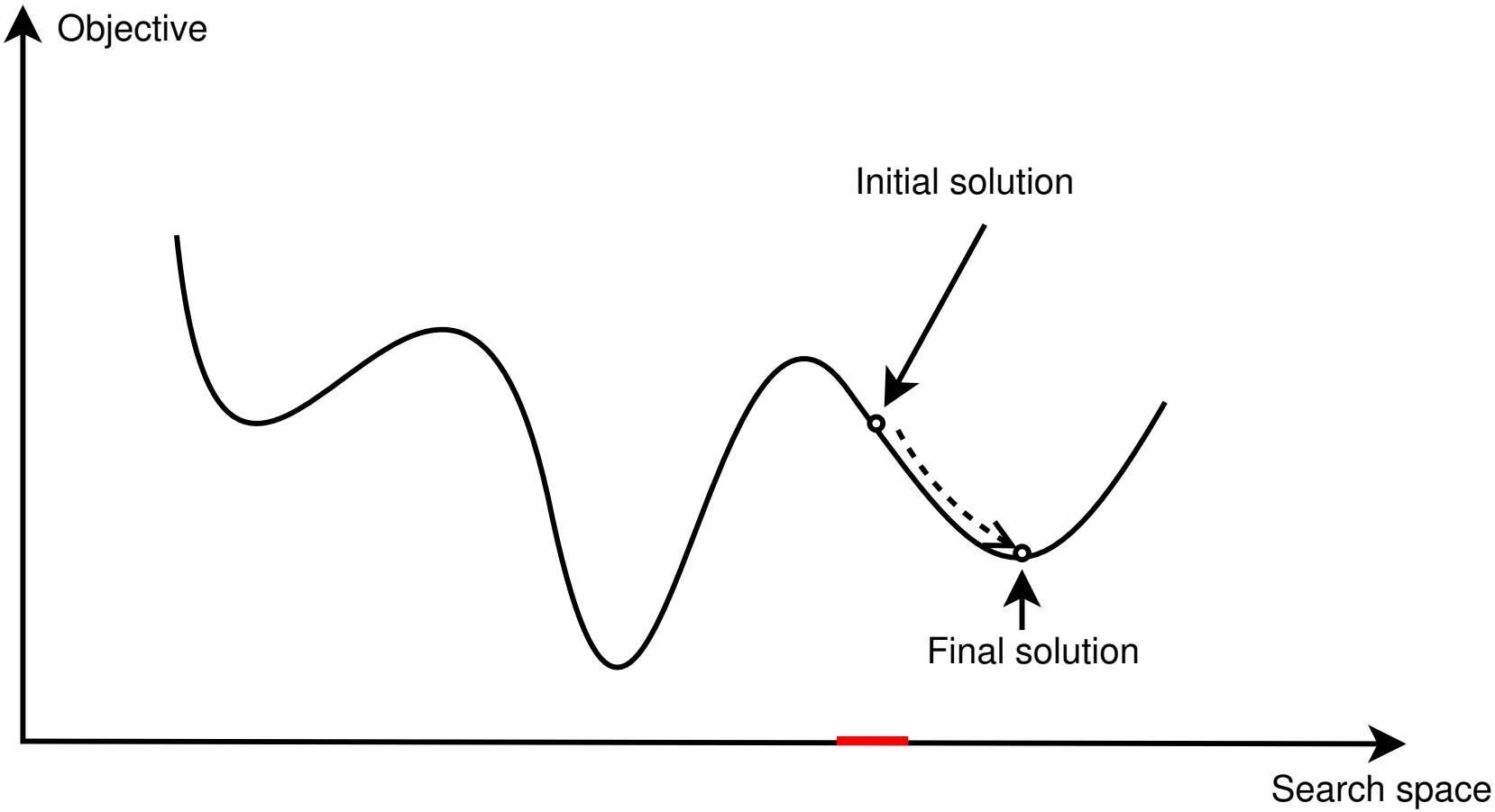
Local Search Algorithmic Template

Input: Initial candidate solution $\mathbf{s}_0$; Output: A high-quality solution

1. $t \leftarrow 0$
2. Generate neighbourhood $N(\mathbf{s}_t)$ of candidate solutions from $\mathbf{s}_t$
3. If $N(\mathbf{s}_t)$ contains no solutions better than $\mathbf{s}_t$, then output $\mathbf{s}_t$ and stop
4. $\mathbf{s}_{t+1} \leftarrow$ a solution in $N(\mathbf{s}_t)$ that is better than $\mathbf{s}_t$
5. $t \leftarrow t + 1$
6. Go to Step 2

Local search progression (p. 122)

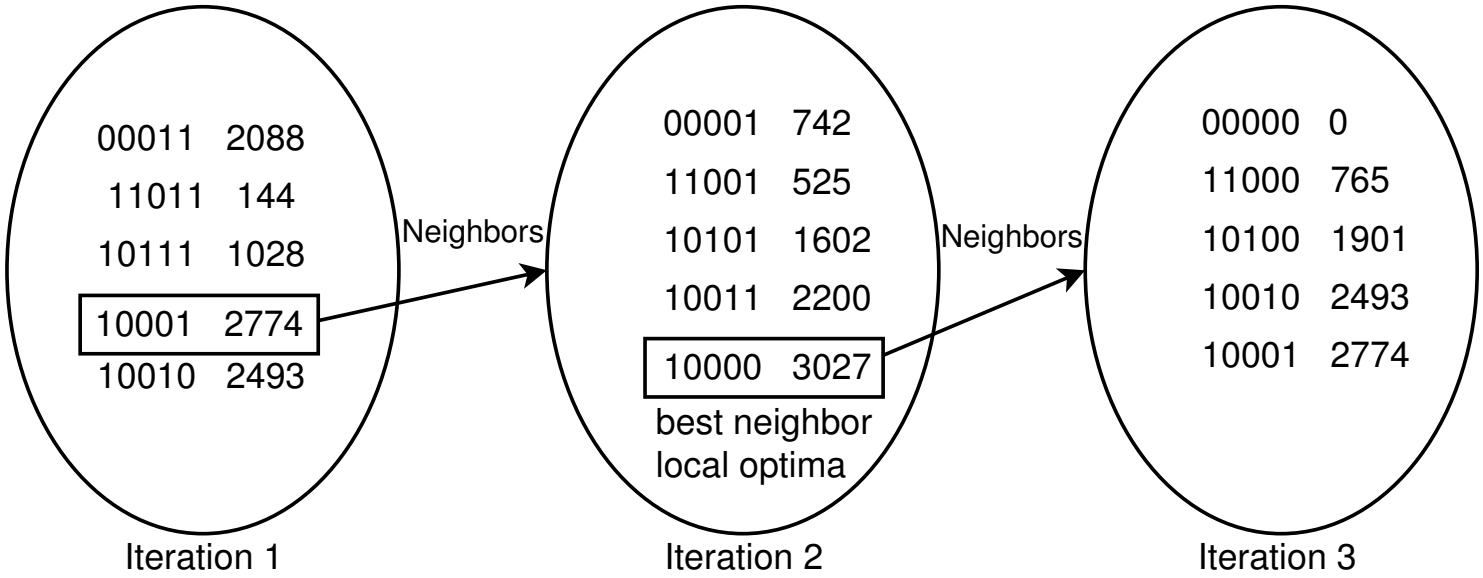
Graphical representation of local search progression:



A simple local search example (p. 121)

Suppose we wish to maximise an unconstrained function of **x**

Suppose **x** is encoded in binary vector form, that we apply local search starting from **x** = 10011 with an objective function value of 1 000, and use single bit complementing as move operator to generate neighbourhoods

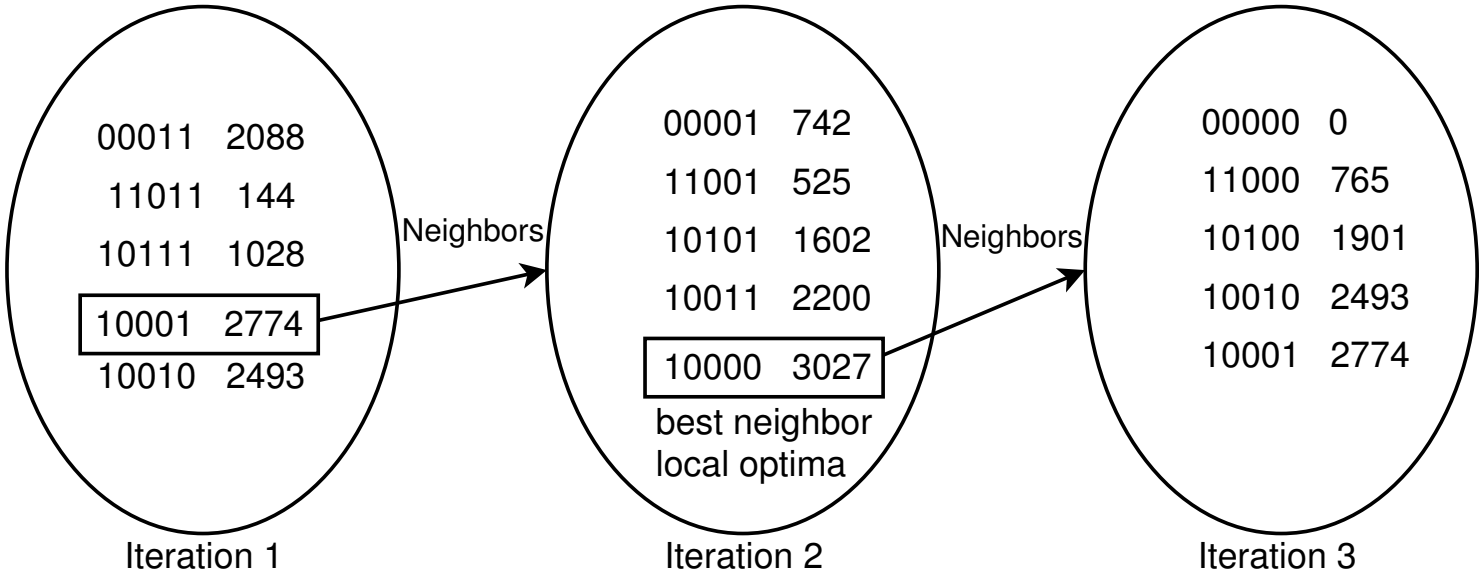


The solution **x** = 10000 is returned, yielding a function value 3 027

Selection of the Neighbour (pp. 123–124)

Methods of neighbour selection (p. 123)

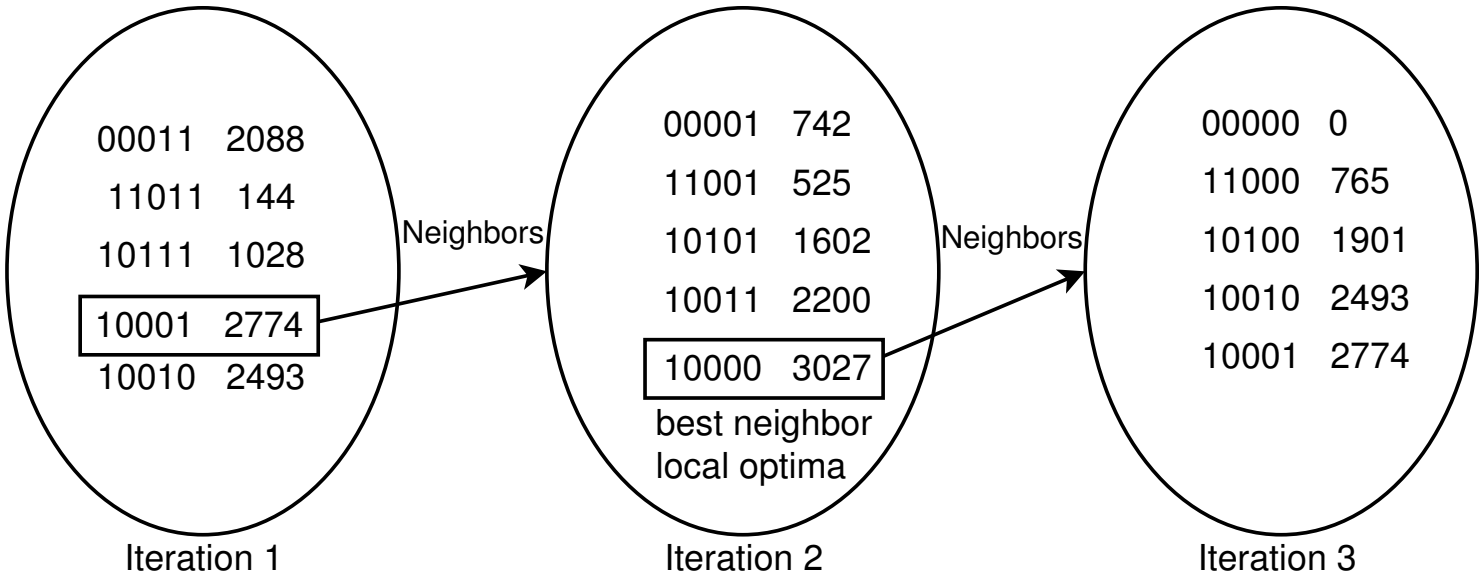
Best improvement. The best neighbour is selected. Hence the neighbourhood is generated exhaustively. This may be prohibitively time-consuming for large neighbourhoods, in which case neighbourhoods may be truncated.



Methods of neighbour selection (p. 123)

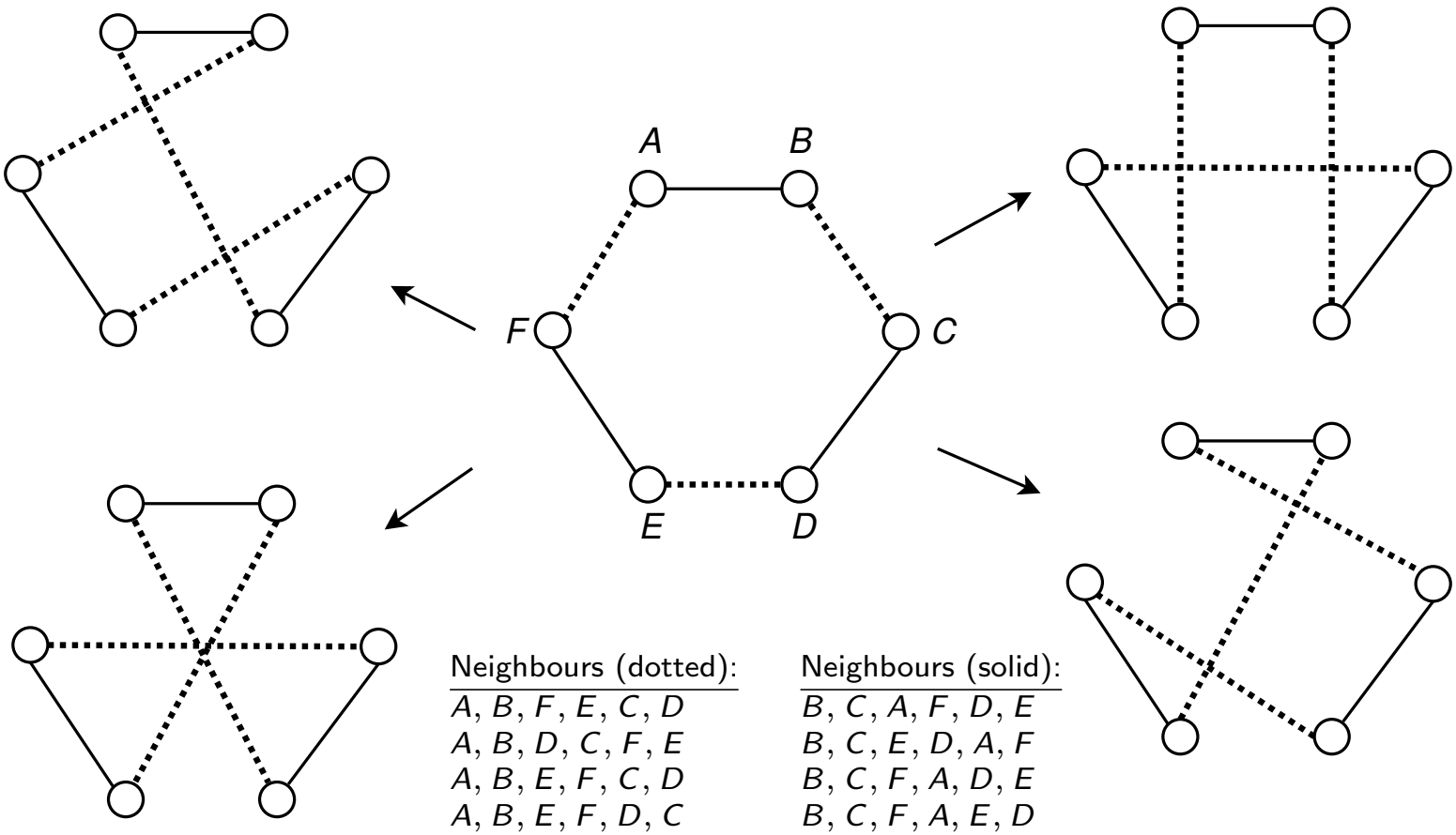
First improvement. The first improving neighbour encountered is selected. The neighbourhood is typically only considered partially, and this is done in a deterministic way.

Random selection. An improving neighbour is randomly selected from the neighbourhood. This may be done very rapidly, but may result in convergence to poor local optima or an excessive number of iterations.



Applying local search to a 6-city TSP using 3-Opt

	A	B	C	D	E	F
A	0	41	26	31	27	35
B	41	0	29	32	40	33
C	26	29	0	25	34	42
D	31	32	25	0	28	34
E	27	40	34	28	0	36
F	35	33	42	34	36	0



Neighbours (dotted):	Neighbours (solid):
A, B, F, E, C, D	B, C, A, F, D, E
A, B, D, C, F, E	B, C, E, D, A, F
A, B, E, F, C, D	B, C, F, A, D, E
A, B, E, F, D, C	B, C, F, A, E, D

Applying local search to a 6-city TSP using 3-Opt

Let's apply local search to a 6-city TSP with distance matrix

	A	B	C	D	E	F
A	0	41	26	31	27	35
B	41	0	29	32	40	33
C	26	29	0	25	34	42
D	31	32	25	0	28	34
E	27	40	34	28	0	36
F	35	33	42	34	36	0

taking $[A, B, C, D, E, F]$ as initial solution and the option of best selection to choose the next current solution

t	$\mathbf{s}^{(t)}$	z	Neighbour	New z	
0	$[A, B, C, D, E, F]$	194	$[A, B, F, E, C, D]$	200	
			$[A, B, D, C, F, E]$	203	
			$[A, B, E, F, C, D]$	215	
			$[A, B, E, F, D, C]$	202	
			$[B, C, A, F, D, E]$	192	
			$[B, C, E, D, A, F]$	190	$\leftarrow$
			$[B, C, F, A, D, E]$	205	
			$[B, C, F, A, E, D]$	193	
1	$[B, C, E, D, A, F]$	190	$[B, C, F, A, E, D]$	193	
			$[B, C, D, E, F, A]$	194	
			$[B, C, A, F, E, D]$	186	$\leftarrow$
			$[B, C, A, F, D, E]$	192	
			$[C, E, B, F, D, A]$	198	
			$[C, E, A, D, B, F]$	199	
			$[C, E, F, B, D, A]$	192	
			$[C, E, F, B, A, D]$	200	
2	$[B, C, A, F, E, D]$	186	$\cdots$	$\cdots$	

Advantages & Disadvantages of Local Search (p. 125)

Advantages & disadvantages of local search (p. 125)

Local search is usually a very easy method to design and implement

It returns fairly good solutions very quickly

These are the main reasons why it is a widely-used optimisation method

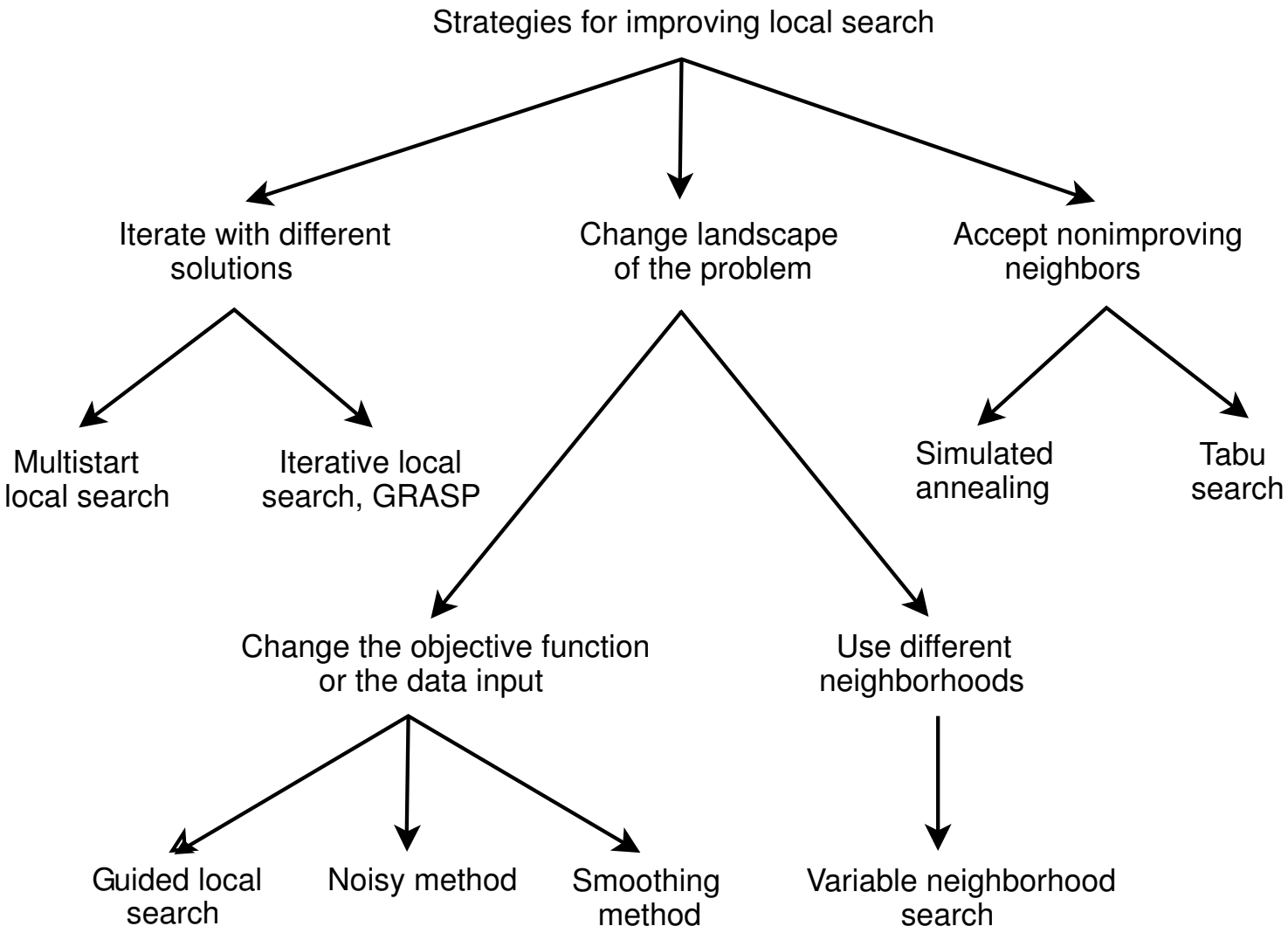
One of its main disadvantages, however, is that it typically converges toward local optima that are not globally optimal

Another is the that algorithm is very sensitive to the particular choice of initial solution

Local search therefore works well if there are not too many local optima in the search space or if the relative qualities of the different local optima are more or less similar

Else a method is required to enable the search to escape local optima

Strategies for escaping from local optima (p. 125)



Escaping from Local Optima (pp. 125–126)
Multistart Local Search

Multistart local search (p. 125)

In multistart local search, a local search is launched k times, where k is a parameter

Therefore, k initial solutions are generated beforehand — one for each of these local searches

The best solution returned by these local searches (called the *incumbent solution*) is recorded and updated as necessary as the local searches progress

Upon termination of all k searches, the incumbent solution is returned

Applying multistart local search to the knapsack problem

$$\begin{aligned} \text{maximise} \quad & z = 18x_1 + 25x_2 + 11x_3 + 14x_4 \\ \text{s.t.} \quad & 2x_1 + 2x_2 + x_3 + x_4 \leq 3 \\ & x_1, x_2, x_3, x_4 \in \{0, 1\} \end{aligned}$$

Use multi-start local search with $k = 2$, initial solutions $[1, 0, 0, 0]$ and $[0, 0, 0, 1]$, single-bit complement moves, and the method of best selection

Start	t	$\mathbf{s}^{(t)}$	z	$\hat{\mathbf{s}}$	$\hat{\mathbf{z}}$	Neighbour	Bit	New z	
1	0	[1, 0, 0, 0]	18	[1, 0, 0, 0]	18	[0, 0, 0, 0]	1	0	
						[1, 1, 0, 0]	2	Infeasible	
						[1, 0, 1, 0]	3	29	
						[1, 0, 0, 1]	4	32	←
1	1	[1, 0, 0, 1]	32	[1, 0, 0, 1]	32	[0, 0, 0, 1]	1	14	
						[1, 1, 0, 1]	2	Infeasible	
						[1, 0, 1, 1]	3	Infeasible	
						[1, 0, 0, 0]	4	18	Restart
2	0	[0, 0, 0, 1]	14	[1, 0, 0, 1]	32	[1, 0, 0, 1]	1	32	
						[0, 1, 0, 1]	2	39	←
						[0, 0, 1, 1]	3	25	
						[0, 0, 0, 0]	4	0	
2	1	[0, 1, 0, 1]	39	[0, 1, 0, 1]	39	[1, 1, 0, 1]	1	Infeasible	
						[0, 0, 0, 1]	2	14	
						[0, 1, 1, 1]	3	Infeasible	
						[0, 1, 0, 0]	4	25	Stop

The approximate solution $\hat{\mathbf{s}} = [0, 1, 0, 1]$ is returned, corresponding to $\hat{z} = 39$

(It is, by accident, also the optimal solution)

Escaping from Local Optima (pp. 146–150)
Iterative Local Search

Iterative local search (p. 146–148)

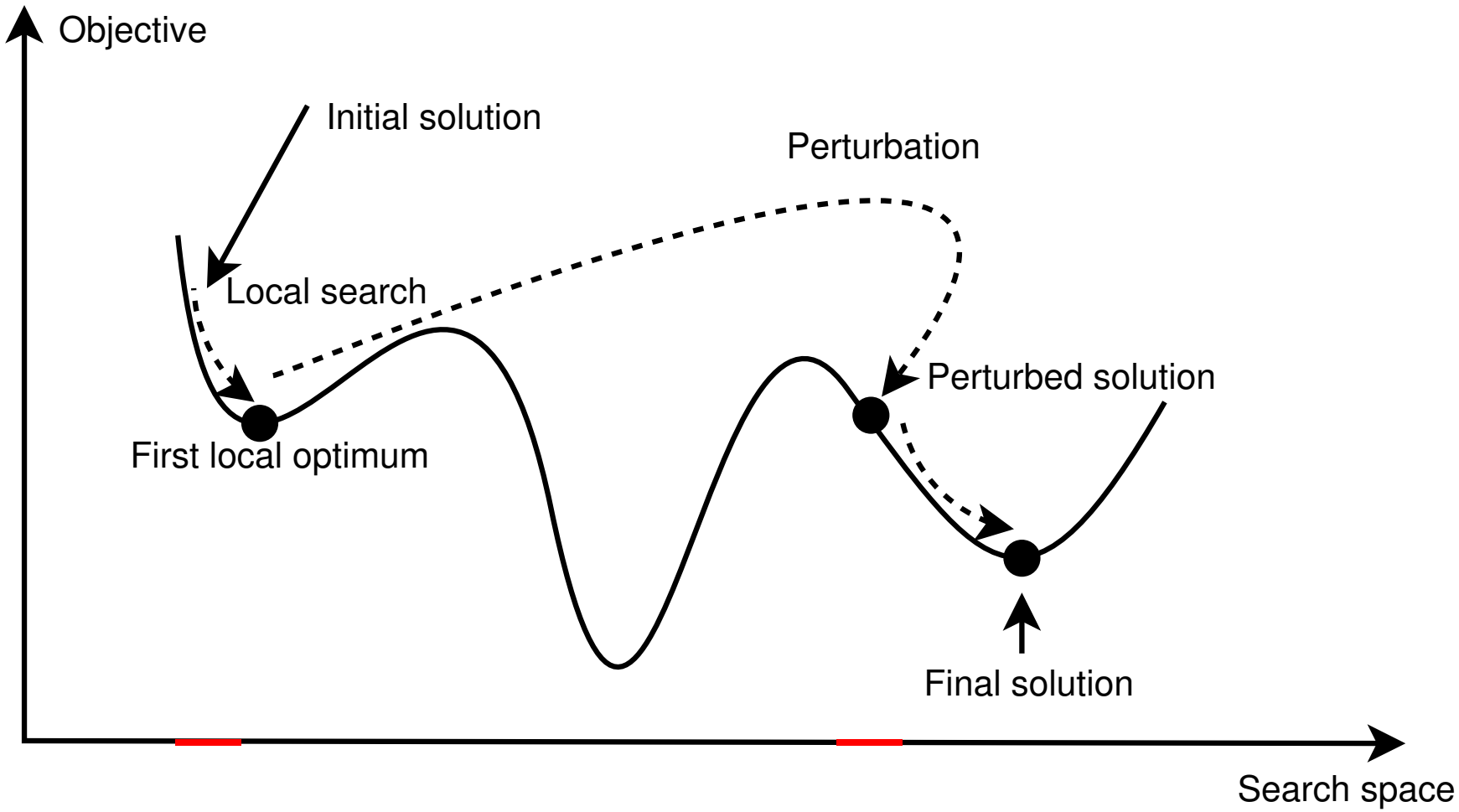
In multi-start local search, the successive initial solutions are chosen randomly (hence they are unrelated to the local optima encountered during the search)

Iterative local search improves upon multi-start local search by successively perturbing the local optimum of a local search according to some *perturbation method* and taking this perturbation as initial solution for a next local search if it satisfies some *acceptance criterion*

If the perturbation is not accepted as the next initial solution, another perturbation is performed

This process is repeated until some stopping condition is met (such as that some maximum total number of perturbations have been performed)

Iterative local search (p. 146–148)



Iterative local search perturbation method (p. 148)

The perturbation method of an iterative local search must be more effective than a random restart approach

For some problem instances, where the landscape is rugged and flat, random restart may, however, be more efficient

The design of an iterative local search should include a comparison of the performance of the random restart local search with that of successive local searches employing biased perturbation to generate initial solutions

The size of the perturbation may be related to the neighbourhood and may be specified in terms of the solution encoding (the number of components of a solution modified)

A too small perturbation may generate cycles during the search (the probability of exploring *other* basins of attraction may be low)

Small perturbations result in faster searches than large perturbations

Too large a perturbation may erase the information about the search memory (rendering useless the good properties of local optima skipped)

The optimal perturbation size depends on the fitness landscape structure and must not exceed the correlation length

Iterative local search perturbation method (p. 149)

Perturbations may be applied globally to the local optimum in which case its size may be

- **Static** The perturbation size is fixed before starting the search
- **Dynamic** The perturbation size is selected dynamically without taking into account the search history
- **Adaptive** The perturbation size is adapted during the search according based on information about the fitness landscape and the search history

The perturbation may also be applied locally to the local optimum by randomly selecting a solution in its neighbourhood

Iterative local search acceptance criteria (p. 146–150)

Together with the perturbation method, the role of the acceptance criterion is to control a trade-off between *intensification* and *diversification*

The extreme in terms of intensification is to accept only improving solutions in terms of the objective function (strong selection)

The extreme in terms of diversification is to accept any solution without regard to its quality (weak selection)

Acceptance criteria adopt some midway between these extremes and come in two classes:

Probabilistic acceptance criteria An example is accepting all improving solutions, but worsening solutions only with some probability

Deterministic acceptance criteria An example is threshold accepting (accepting any solution which is no worse by some margin than the local optimum of the previous local search)

Applying iterative local search to the knapsack problem

$$\begin{aligned} \text{maximise} \quad & z = 18x_1 + 25x_2 + 11x_3 + 14x_4 \\ \text{s.t.} \quad & 2x_1 + 2x_2 + x_3 + x_4 \leq 3 \\ & x_1, x_2, x_3, x_4 \in \{0, 1\} \end{aligned}$$

Let use iterative local search with:

- Initial solution $[0, 0, 1, 0]$,
- Single-bit complement moves during each local search,
- Static perturbation with a perturbation size of two bits,
- Accepting solutions no more than $33.\dot{3}\%$ worse than the local optimum,
- Performing a maximum of three perturbations as stopping criterion

Search	t	$\mathbf{s}^{(t)}$	z	$\hat{\mathbf{s}}$	$\hat{\mathbf{z}}$	Neighbour	Bit	New z	
1	0	[0, 0, 1, 0]	11	[0, 0, 1, 0]	11	[1, 0, 1, 0]	1	29	
						[0, 1, 1, 0]	2	36	←
						[0, 0, 0, 0]	3	0	
						[0, 0, 1, 1]	4	25	
1	1	[0, 1, 1, 0]	36	[0, 1, 1, 0]	36	[1, 1, 1, 0]	1	Infeasible	
						[0, 0, 1, 0]	2	11	
						[0, 1, 0, 0]	3	25	
						[0, 1, 1, 1]	4	Infeasible	Perturbation 1

Applying iterative local search to the knapsack problem

Search	t	$\mathbf{s}^{(t)}$	z	$\hat{\mathbf{s}}$	$\hat{\mathbf{z}}$	Neighbour	j	New z	
1	0	[0, 0, 1, 0]	11	[0, 0, 1, 0]	11	[1, 0, 1, 0]	1	29	
						[0, 1, 1, 0]	2	36	←
						[0, 0, 0, 0]	3	0	
						[0, 0, 1, 1]	4	25	
1	1	[0, 1, 1, 0]	36	[0, 1, 1, 0]	36	[1, 1, 1, 0]	1	Infeasible	
						[0, 0, 1, 0]	2	11	
						[0, 1, 0, 0]	3	25	
						[0, 1, 1, 1]	4	Infeasible	Perturbation 1
		Perturbation 1: [0, 1, 1, 0] ↦ [1, 1, 0, 0] . . . Infeasible							
		Perturbation 2: [0, 1, 1, 0] ↦ [0, 0, 1, 1] . . . z = 25							
2	0	[0, 0, 1, 1]	25	[0, 1, 1, 0]	36	[1, 0, 1, 1]	1	Infeasible	
						[0, 1, 1, 1]	2	Infeasible	
						[0, 0, 0, 1]	3	14	
						[0, 0, 1, 0]	4	11	Perturbation 3
		Perturbation 3: [0, 0, 1, 1] ↦ [0, 1, 0, 1] . . . z = 39							
3	0	[0, 1, 0, 1]	39	[0, 1, 0, 1]	39	[1, 1, 0, 1]	1	Infeasible	
						[0, 0, 0, 1]	2	14	
						[0, 1, 1, 1]	3	Infeasible	
						[0, 1, 0, 0]	4	25	Stop

The approximate solution $\hat{\mathbf{s}} = [0, 1, 0, 1]$ is returned, corresponding to $\hat{z} = 39$